

ALGORITMOS Y PROGRAMACION
Informe Buenas Practicas

Carlos Andres Beltran Zuñiga

Fundación Universitaria

Compensar Algoritmos y

Programación 23101A

María Fernanda Hernández Vargas

Bogota.D.C

29 de mayo de 2026

Contenidos

temáticos Sentencias de decisión simples y anidadas

Sentencias de decisión múltiple (casos)

Sentencias de repetición

Uso especial de variables: conteo, sumatoria, bandera

Sentencia de repetición condicionada al final

Sentencias de repetición condicionadas al comienzo

Iteraciones ascendentes, descendentes

Modelamiento y construcción de programas a partir de algoritmos que involucran control de flujo de ejecución con sentencias de repetición

Introducción a estructuras de datos simples (arreglos y listas)



INTRODUCCION

El presente trabajo tiene como finalidad la implementación de nuevos conceptos para realizar lo que se denomina *código limpio*, para mejorar y estructurar de una manera más dinámica y mas organizada los programas o proyectos que realizaremos. La importancia de un buen código limpio se generaliza en que hace parte de buenas practicas para el programador o cliente que desarrolle o se le haga llegar dicho algoritmo. Dicho esto, un código bien organizado mejorara la eficiencia de corrección de errores o de actualización del programa que soluciona hasta el mas simple detalle. Es de entender que los programadores no vemos por igual el mismo camino para solucionar un problema y es por esto que el código que desarrollemos sea entendible y practico.

Para realizar lo anterior mencionado se recurre a herramientas y conceptos para implementar nuestro código y cumplir con el objetivo principal. El uso de funciones y programación por bloques o lo conocido como modularizarían ya que permite repartir tareas haciendo y como veremos más adelante permitirá encontrar y solucionar problemas o actualizaciones de reajuste del programa además de una organización de este más amena al usuario o programador. Adicional a esto es importante conocer las herramientas que podemos usar como funciones recursivas que mejoraran nuestro código haciéndolo más dinámico y sin extenderse.



Justificación

En este proyecto se requiere un programa para un gestor de tareas "To do list", es un buen ejercicio para aplicar los temas de modularizarían con funciones, e independizar tareas dentro del programa porque el problema lo permite ya que las opciones como agregar tareas buscar, mostrar, controlar tareas pendientes y marcarlas etc, se pueden programar con funciones independientes lo que hace el programa más organizado y legible. además, que facilita la practica de fundamentales en programación como la generación de listas, condicionales ciclos o bucles, funciones recursivas e incluso el monitor de errores en datos de entrada, así bien un gestor de tareas es un buen aprendizaje a futuro para solución e implementación de los temas mencionados para incentivar y promover el uso de buenas prácticas y una estructura más limpia de programación (clean code).

Identificación y explicación de módulos

A continuación, se explicará las funciones empleadas para el desarrollo de la actividad en este caso “gestor de tareas”.

Como principio debemos crear una nueva clase para generar la lista que deseamos.

```
class Tarea{  
    String tareas;  
    boolean completada;  
  
    public Tarea(String tareas, boolean completada){  
        this.tareas = tareas;  
        this.completada = completada;  
    }  
}
```

Dicha clase tendrá una tarea tipo string y otro tipo boolean. Con esto se guardarán dos cosas en la lista. Una descripción y si esta completada o no (false o true).

La condición “this” es para guardar cada parámetro y diferenciarlo de cada uno para que al llenar la lista se guarden por separado. La creación de una clase en vez de definir (arraylist) lista por lista facilita el no crear más código para definir las, con la clase queda todo definido ahí y sería más sencillo agregar otra instancia a la lista.

Función obtener tareas

```
public static List<String> obtenerTareas(){
    List<String> listaTareas = new ArrayList<>();
    for(int i=0; i<tareas.size(); i++){ // for para
recorrer las listas tareas y completada,
        //para crear una nueva lista organizada.
        if(tareas.get(i).completada==false){
            //si la tarea esta pendiente, se agrega a la
lista de tareas con el estado marcado como [].

            listaTareas.add(tareas.get(i).tareas + "\t\t[]\n");
        }
        else{//si la tarea esta completada, se agrega a
la lista de tareas con el estado marcado como [x].

            listaTareas.add(tareas.get(i).tareas + "\t\t[x]\n");
        }
    }
    return listaTareas;
}
```

Función para obtener las tareas o descripción de las actividades que el usuario desea ingresar y/o realizar y su estado (pendiente o completada), para luego presentarla o imprimirla de una manera organizada. Esta función recorre por medio de un for las listas declaradas, (tareas y completada), para crear una nueva lista, no recibe parámetros, pero retorna una lista tipo String con las tareas y su estado marcado con [x] si esta completada [] si está pendiente.

Función mostrar tareas

Parámetros que recibe: *listaTareas*.

```
public public static List<Tarea> agregarTarea(List<Tarea> listaTareas,boolean
completada){
    Scanner scanner = new Scanner(System.in);
    do{
        System.out.println("Describe la nueva tarea: ");
        String nuevaTarea = scanner.nextLine();
        listaTareas.add(new Tarea(nuevaTarea, false));
        System.out.println("Tarea agregada exitosamente\n");
        System.out.println("Desea agregar otra tarea? (s/n)");
        respuesta= scanner.next();
        scanner.nextLine();
    } while (respuesta.equalsIgnoreCase("s"));//
    el ciclo termina si el usuario no desea
    if (respuesta.equalsIgnoreCase("n")){
        System.out.println("Desea volver al menu principal? (s/n)");
        respuesta2=scanner.next();
        if (respuesta2.equalsIgnoreCase("s")){
            continuar=true;
        }
        else {
            System.out.println("Buen dia\n");
            continuar=false;
        }
    }

    return listaTareas;
}
```

función para mostrar las tareas a realizar y su estado actual. Esta función recibe como parámetro la listaTareas creada a partir de la función anteriormente explicada y mediante un **for** la recorre para organizarla y mostrarla en pantalla, para mostrarlas, no retorna nada cuando es llamada, solo muestra la lista de tareas a realizar y su estado, también pregunta al usuario si desea volver al menú principal o salir.



Funcion agregar tareas

Parametros: (ListaTareas, Completada).

```
public static List<Tarea> agregarTarea(List<Tarea> listaTareas, boolean
completada){
    Scanner scanner = new Scanner(System.in);
    do{
        System.out.println("Describe la nueva tarea: ");
        String nuevaTarea = scanner.nextLine();
        listaTareas.add(new Tarea(nuevaTarea, false));
        System.out.println("Tarea agregada exitosamente\n");
        System.out.println("Desea agregar otra tarea? (s/n)");
        respuesta= scanner.next();
        scanner.nextLine();
        } while (respuesta.equalsIgnoreCase("s")); //el ciclo
termina si el usuario no desea
        if (respuesta.equalsIgnoreCase("n")){
            System.out.println("Desea volver al menu principal?
(s/n)");
            respuesta2=scanner.next();
            if (respuesta2.equalsIgnoreCase("s")){
                continuar=true;
            }
            else {
                System.out.println("Buen dia\n");
                continuar=false;
            }
        }

        return listaTareas;
    }
}
```

función para agregar una tarea nueva y su estado a la lista de tareas a realizar. Esta función recibe como parámetros la lista de tareas y la lista de estados.

Con ayuda de la clase scanner y como primera instancia se pide al usuario ingresar por teclado una nueva tarea, esta se agrega a la lista de tareas (`listaTareas.add`) y se agrega un estado "false" a la lista de estados *completada* en la misma posición para indicar que está pendiente.

Esta función retorna la lista de tareas actualizada, a su vez dicha lista se mostrará cuando el usuario vuelva al menú principal y se pida mostrar tareas en la opción 1, también pregunta al usuario si desea agregar otra tarea o volver al menú principal o salir. Esta función emplea un condicional "do while" con el objetivo de que se repita el ciclo y permita regresar al menú principal para seleccionar una opción o agregar más de una tarea a la lista en la misma opción.

función marcar completada

parámetros: (listatareas, completada).

```
public static void marcarCompletada(List<Tarea> listaTareas, Boolean
completada){
    Scanner scanner = new Scanner(System.in);
    do{
        System.out.println("Ingrese el numero de la tarea que desea
marcar como completada: ");
        int numeroTarea = scanner.nextInt();
        try{//try catch para manejar el error de ingresar un numero de
tarea inexistente o invalido.
            listaTareas.get(numeroTarea - 1).completada = true;//set
cambia el valor de la posicion numeroTarea-1 en la lista completada a
true, indicando que la tarea esta completada.
            System.out.println("Tarea marcada como completada\n");
        }
        catch (IndexOutOfBoundsException e){//catch para manejar el error
de ingresar un numero de tarea inexistente o invalido.
            System.out.println("Error: Numero de tarea invalido o
no existe\n");
        }
        System.out.println("Desea marcar otra tarea como completada?
(s/n)");
        respuesta= scanner.next();
        scanner.nextLine();
    } while (respuesta.equalsIgnoreCase("s"));//el ciclo termina si
el usuario no desea
    System.out.println("Desea volver al menu principal? (s/n)");
    respuesta=scanner.next();
    if (respuesta.equalsIgnoreCase("s")){
        continuar=true;
    }
    else {
        System.out.println("Buen dia\n");
        continuar=false;
    }
}
```



función que permite marcar la tarea dentro del listado que tengamos en el momento como completada es decir cambiar su estado de false a true. Nuevamente se llama a clase scanner y con el fin de que el usuario pueda ingresar la posición por teclado y esta será guardada en una variable y con ello buscaremos dentro de la lista de tareas.

try catch: para esta función se pidió usar un try cath para manejar posibles errores de entrada. En este caso si el usuario digita por error un numero invalido o un dato erróneo el programa saltara un mensaje de error. Por el contrario, si no hay error el programa buscara en la posición menos 1 (`listaTareas.get(numeroTarea - 1).completada = true`) ya que recordemos que en listas las posiciones empiezan desde cero, y en dicha posición cambiara su estado por un true para aclarar que la actividad fue realizada.

Finalizamos el try catch con un (`listaTareas.get(numeroTarea - 1).completada = true`), para manejar el error de ingresar un dato erróneo. Y todo en un do while para darle continuidad al programa o finalizarlo.

**función recursiva: función contar tareas pendientes.**

Parametros: (índice, tareas)

```
public static int contarTareasPendientes(int indice, List<Tarea>
tareas){
    if (indice== tareas.size()){//caso base: cuando el indice llega
a ser igual al tamaño de la lista completadas.
        return 0;// retorna 0, indicando que no hay tareas
pendientes.
    }

    if (!tareas.get(indice).completada) {
        // Si está pendiente, sumamos 1 y seguimos con el siguiente
índice
        return 1 + contarTareasPendientes(indice + 1, tareas);
    }

    else {
        return contarTareasPendientes(indice + 1, tareas);// Si está
completada, solo seguimos con el siguiente índice sin
//sumar nada.
    }

}
```

Para este punto en específico se resolvió mediante la creación de una función recursiva, es decir una función q se llama así misma dentro para resolver un problema.

Para lograr esto debemos crear el caso base partiendo de que la función tiene como parámetros un índice=0 ya definido el cual servirá para ubicar la posición actual y listaTareas: el caso base es la condición que detendrá el ciclo de la recursividad. En este caso.

```
if (indice== tareas.size())
```

si el índice es igual al tamaño o la ultima posición del listado el ciclo finaliza sin embargo al inicio como el índice es 0, esto significa que estaremos ubicados en la primera posición y pasaremos a la siguiente condición.

```
if (!tareas.get(indice).completada)
```

primero tareas.get(0). Completada buscara en la posicion inicial 0 de la lista de tareas el valor booleano en este caso de completada de dicha posicion y "!" al usar este signo estamos diciendo "NO" es decir es negativa o false ("no esta completada"), entonces entra a hacer la siguiente operación.

```
return 1 + contarTareasPendientes(indice + 1, tareas);
```

que significa que, si encuentra una pendiente, retorna 1, y además suma a este retorno lo que resulte del llamado nuevamente de la función.

contarTareasPendientes(indice + 1, tareas)

donde el índice esta vez no será 0 si no que será $\text{indice}=0+1$; con esto se repetirá el ciclo hasta que se aplique el caso base de la primera parte.

Sin embargo, si cuando en dicha posición se comprueba que la tarea esta completada.

return contarTareasPendientes(indice + 1, tareas);

no sumara nada y sigue con el siguiente índice.

Ejemplos de ejecución:

1. El programa mostrara un menú principal

```
Listening on 52241
User program running
seleccione una opcion:
1. Ver tareas
2. Agregar tarea
3. Marcar tarea como completada
4. Mostrar total de tareas pendientes
5. Salir
```

2. Seleccionamos opción 1

```
Listening on 52241
User program running
seleccione una opcion:
1. Ver tareas
2. Agregar tarea
3. Marcar tarea como completada
4. Mostrar total de tareas pendientes
5. Salir
1

1 - Hacer trabajos          []
2 - Estudiar inglés        []
3 - Hacer ejercicio        []

Desea volver al menu principal? (s/n)
```

Nos muestra la lista que hemos creado predeterminadamente en la cual observaremos 3 tareas las cuales siguen pendientes “[]”, la función mostrar tareas es la que nos define y nos organiza dicha lista para que el usuario la vea de esta manera. Tenemos la facilidad de volver al menú anterior.

3. Seleccionamos opción 2 “agregar tareas”



```
Describe la nueva tarea:
ir a hacer mercado

Tarea agregada exitosamente

Desea agregar otra tarea? (s/n)
s

Describe la nueva tarea:
tocar guitarra

Tarea agregada exitosamente

Desea agregar otra tarea? (s/n)
n

Desea volver al menu principal? (s/n)
s

seleccione una opcion:
1. Ver tareas
2. Agregar tarea
3. Marcar tarea como completada
4. Mostrar total de tareas pendientes
5. Salir
```

Permite agregar una tarea describiéndola, y pregunta al usuario si desea seguir agregando aún más, si el caso es que no, podemos volver al menú principal.

4. Opción 3 “marcar tarea como terminada.

Esta opción pide al usuario ingresar el numero de la tarea a marcar como completada. Una vez marcada puede volver al menú principal para seleccionar la opción de mostrar tareas y se mostraran las tareas marcadas.



Ingrese el numero de la tarea que desea marcar como completada:

5

Tarea marcada como completada

Desea marcar otra tarea como completada? (s/n)

s

Ingrese el numero de la tarea que desea marcar como completada:

2

Tarea marcada como completada

Desea marcar otra tarea como completada? (s/n)

n

Desea volver al menu principal? (s/n)

seleccione una opcion:

1. Ver tareas
2. Agregar tarea
3. Marcar tarea como completada
4. Mostrar total de tareas pendientes
5. Salir

1

1 - Hacer trabajos ☐

2 - Estudiar inglés ☒

3 - Hacer ejercicio ☐

4 - ir a hacer mercado ☐

5 - tocar guitarra ☒

Desea volver al menu principal? (s/n)

|

5. La opción 4 “mostrar total de tareas pendientes”.

seleccione una opcion:

1. Ver tareas

2. Agregar tarea

3. Marcar tarea como completada

4. Mostrar total de tareas pendientes

5. Salir

4

Las tareas pendientes son: 3

Desea volver al menu principal? (s/n)

Cuenta las tareas que están pendientes “[]”, de acuerdo con el listado general.

Todas las opciones como se pueden apreciar tienen la posibilidad de retornar al menú principal y en algunos casos de repetir el proceso bien se para llenar o marcar alguna tarea.

Código java

Código para recrear una "To do List" o un gestor de tareas, mostrará un menú y permitirá al usuario agregar tareas mostrarlas, marcar las pendientes y las que no.

```
import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;
import java.util.Scanner;

/**creacion de una clase llamada tarea cuya funcion es definir las lista que
manejaremos en el programa una de tipo string para almacenar las tareas a
realizar y otra de tipo boolean para almacenar el estado de cada tarea,
 * esta clase tiene un constructor que recibe como parametros una tarea y su
estado, y asigna estos parametros a las variables de la clase, esta clase se
utiliza para crear objetos de tipo tarea que se almacenan
 * en una lista de tareas,
 */
class Tarea{
    String tareas;
    boolean completada;

    public Tarea(String tareas, boolean completada){
        this.tareas = tareas;//asignacion del parametro tarea a la variable de
la clase tareas.
        this.completada = completada;//asignacion del parámetro completada a
la variable de la clase completada.
    }
}

public class mi_gestor_tareas{
    static int options;// variable para opciones del menu
    static Scanner Seleccion = new Scanner(System.in);
    static boolean continuar = true; // variable para controlar el bucle
del menu

    static String respuesta;
    static String respuesta2;
    static List<Tarea> tareas = new ArrayList<>(Arrays.asList(
        new Tarea("Hacer trabajos", false),
        new Tarea("Estudiar inglés", false),
        new Tarea("Hacer ejercicio", false)
    ));    /*+ definimos la lista de tareas a realizar, esta lista se
inicializa con tres tareas pendientes, cada tarea es un objeto de tipo Tarea
que contiene una descripción de la tarea y su estado de completada o
pendiente. */
```



```
////////////////////////////////////
/**funcion para obtener las tareas a realizar y su estado.
Esta funcion recorre las listas declaradas, tareas y completada, para crear
una nueva lista organizada, no recibe parametros, pero retorna una lista tipo
string con las tareas y su estado marcado con [x] si esta completada
o [] si esta pendiente.*/

    public static List<String> obtenerTareas(){
        List<String> listaTareas = new ArrayList<>();
        for(int i=0; i<tareas.size(); i++){ /** for para recorrer
las listas tareas y completada,
        para crear una nueva lista organizada.*/
            if(tareas.get(i).completada==false){
                //si la tarea esta pendiente, se agrega a la lista de
tareas con el estado marcado como [].
                listaTareas.add(tareas.get(i).tareas + "\t\t[]\n");
            }
            else{//si la tarea esta completada, se agrega a la
lista de tareas con el estado marcado como [x].
                listaTareas.add(tareas.get(i).tareas +
"\t\t[x]\n");
            }
        }
        return listaTareas;
    }
}
```

```
//opción 1: Ver tareas
//función para mostrar las tareas a realizar y su estado.
Esta función recibe como parametro la listaTareas para mostrarlas porque es la
lista que contiene los datos que mostraremos en pantalla, no retorna nada
cuando es llamada, muestra la lista de tareas a realizar y su estado, también
pregunta al usuario si desea volver al menu principal o salir. */
    public static void mostrarTareas(List<String> listaTareas){
        Scanner scanner = new Scanner(System.in);
        int posicion=0;
        for(int i=0; i<listaTareas.size(); i++){/** for para recorrer la lista
de tareas a mostrar,
        se muestra el numero de la tarea y su descripcion con su estado.
        ademas el contador nos sirve para mostrar la posicion de la tarea en
la lista*/
            posicion++;
            System.out.println(posicion + " - " + listaTareas.get(i));
        }
        System.out.println("Desea volver al menu principal? (s/n)");
        respuesta=scanner.next();
        if (respuesta.equalsIgnoreCase("s")){
            continuar=true;
        }
        else {
```



```
        System.out.println("Buen dia\n");
        continuar=false;
    }
}
```

//opcion 2: Agregar tarea

/**función para agregar una tarea nueva y su estado a la lista de tareas a realizar. * esta función recibe como parámetros la lista de tareas y la lista de estados. pide al usuario * ingresar por teclado una nueva tarea, la agrega a la lista de tareas y se agrega un estado "false"* a la lista de estados en la misma posición para indicar que está pendiente, retorna la lista de tareas actualizada, * esta lista actualizada se muestra al usuario si se regresa al menú principal y se selecciona la opción 1 de ver tareas,* también pregunta al usuario si desea agregar otra tarea o volver al menú principal o salir.*

```
    public static List<Tarea> agregarTarea(List<Tarea> listaTareas,boolean
completada){
        Scanner scanner = new Scanner(System.in);
        do{
            System.out.println("Describe la nueva tarea: ");
            String nuevaTarea = scanner.nextLine();
            listaTareas.add(new Tarea(nuevaTarea, false));
            System.out.println("Tarea agregada exitosamente\n");
            System.out.println("Desea agregar otra tarea? (s/n)");
            respuesta= scanner.next();
            scanner.nextLine();
        } while (respuesta.equalsIgnoreCase("s")); //el ciclo termina
si el usuario no desea
        if (respuesta.equalsIgnoreCase("n")){
            System.out.println("Desea volver al menu principal?
(s/n)");

            respuesta2=scanner.next();
            if (respuesta2.equalsIgnoreCase("s")){
                continuar=true;
            }
            else {
                System.out.println("Buen dia\n");
                continuar=false;
            }
        }

        return listaTareas;
    }
}
```



```
//opcion 3: Marcar tarea como completada
/**
 * funcion que marca una de las tareas de la lista como completada, esta
 * función recibe dos parámetros listaTareas y completada.
 * Por teclado ingresa el usuario el numero de la tarea que desea marcar como
 * completada, esta función cambia el valor de la posición numeroTarea-1 en la
 * lista completada a true, indicando que la tarea esta completada, no retorna
 * nada pero si modifica la lista completada en su estado, esta lista actualizada
 * se muestra al usuario si se regresa al menú principal y se selecciona la
 * opción 1 de ver tareas, sin embargo como se pide, se realizó el llamado del
 * try catch para manejar el error de ingresar un numero de tarea inexistente o
 * invalido,
 * en este caso se muestra un mensaje de error y se pregunta al usuario si
 * desea marcar otra tarea como completada o volver al menú principal o salir.
 */
public static void marcarCompletada(List<Tarea> listaTareas, Boolean
completada){
    Scanner scanner = new Scanner(System.in);
    do{
        System.out.println("Ingrese el numero de la tarea que desea marcar
como completada: ");
        int numeroTarea = scanner.nextInt();
        try{//try catch para manejar el error de ingresar un numero de tarea
inexistente o invalido.
            listaTareas.get(numeroTarea - 1).completada = true;/** set cambia
el valor de la posicion numeroTarea-1 en la lista completada a true,
            indicando que la tarea esta completada.*/
            System.out.println("Tarea marcada como completada\n");
        }
        catch (IndexOutOfBoundsException e){//catch para manejar el error de
ingresar un numero de tarea inexistente o invalido.
            System.out.println("Error: Numero de tarea invalido o no
existe\n");
        }
        System.out.println("Desea marcar otra tarea como completada? (s/n)");
        respuesta= scanner.next();
        scanner.nextLine();
    } while (respuesta.equalsIgnoreCase("s"));//el ciclo termina si el
usuario no desea
    System.out.println("Desea volver al menu principal? (s/n)");
    respuesta=scanner.next();
    if (respuesta.equalsIgnoreCase("s")){
        continuar=true;
    }
    else {
        System.out.println("Buen dia\n");
        continuar=false;
    }
}
```

```
//opcion 4: Mostrar total de tareas pendientes
/**
 * funcion para contar el número de tareas pendientes, esta función recibe
 como parámetros un índice y la lista completada, esta función utiliza
 recursividad para recorrer la lista completada y contar el número de tareas
 pendientes, el caso base es cuando el índice llega a ser igual al tamaño de la
 lista completada, en este caso se retorna 0, indicando que no hay tareas
 pendientes, si el valor en la posición del índice es false, se suma 1 y se
 llama recursivamente a la función con el siguiente índice, si el valor en la
 posición del índice es true, se llama recursivamente a la función con el
 siguiente indice sin sumar nada, esta función retorna el número total de
 tareas pendientes.
 */
public static int contarTareasPendientes(int indice, List<Tarea> tareas){
    if (indice== tareas.size()){//caso base: cuando el indice llega a ser
igual al tamaño de la lista completadas.
        return 0;// retorna 0, indicando que no hay tareas pendientes.
    }

    if (!tareas.get(indice).completada) {
        /** Si está pendiente, retorna 1 más el resultado de contar las tareas
pendientes a partir del siguiente índice+1.
        * Esto suma 1 por la tarea pendiente actual y continúa contando las
siguientes tareas. hasta que se alcance el caso base.
        */
        return 1 + contarTareasPendientes(indice + 1, tareas);
    }

    else {
        return contarTareasPendientes(indice + 1, tareas);// Si está
completada, solo seguimos con el siguiente índice sin
//sumar nada.

    }

}
```



```
//opcion 5: Salir del programa
    public static void Salir(){
        System.out.println("Buen dia\n");
        continuar=false;
    }

    public static void main(String[] args) {

        do
        {
            obtenerTareas();

            // seleccion de opciones:
            System.out.println("seleccione una opcion: ");
            System.out.println("1. Ver tareas");
            System.out.println("2. Agregar tarea");
            System.out.println("3. Marcar tarea como completada");
            System.out.println("4. Mostrar total de tareas pendientes");
            System.out.println("5. Salir");
            options = Seleccion.nextInt();

            switch (options) {/** switch para seleccionar la opción del menú,
dependiendo de la opción seleccionada se llama a la función correspondiente,
cada caso corresponde a una opción del menú, si se selecciona una
opción valida se ejecuta la función correspondiente, si se selecciona una
opción no valida se muestra un mensaje de error y se vuelve a mostrar el menú.
*/

                case 1:

                    ///opcion 1: Agregar estudiantes
                    mostrarTareas(obtenerTareas());

                    break;

                    ///opcion 2: Mostrar estudiantes*/
                case 2:

                    agregarTarea(tareas, tareas.get(0).completada);
                    break;

                    ///opcion 3: Marcar tarea como completada
                case 3:

                    marcarCompletada(tareas, tareas.get(0).completada);
                    break;

                    ///opcion 4: Mostrar total de tareas pendientes
                case 4:
```



```
Scanner scanner = new Scanner(System.in);
int pendientes = contarTareasPendientes(0, tareas);
System.out.println("Las tareas pendientes son: " +
pendientes);

System.out.println("Desea volver al menu principal? (s/n)");
respuesta=scanner.next();
if (respuesta.equalsIgnoreCase("s")){
    continuar=true;
}
else {
    continuar=false;
}

break;

//opcion 5: Salir del programa
case 5:
    Salir();
    break;
//opcion cuando se digita un numero que no esta en el menu:
Opcion no valida
default: {
    System.out.println("Opcion no valida\n");
    continuar = true;
}
} while(continuar==true);
}

}
```



Conclusiones técnicas

Una reflexión del uso de programación por módulos es que nos permite crear una estructura mas ordenada más limpia de código y es más sencillo entender el programa porque si bien el creador del código puede tener claridad de que hace cada parte, es muy probable que el código sea usado por alguien mas y no logre comprenderlo y no habrá espacio para ambigüedades en el entendimiento del programa, adicional emplear una modularización en el código permite realizar de manera más efectiva la solución de errores en el mismo y poder hacer mantenimientos para mejorarlo.

Por otro lado el uso de herramientas como funciones recursivas nos brinda un desarrollo mas organizado ya que dichas funciones pueden trabajar por si mismas para cumplir un único objetivo ayudando a comprender mejor los conceptos de recursividad y entender mejor los problemas que pueden dividirse en pasos repetitivos sin embargo debe ser bien documentado y explicado porque puede ser algo mas complejo que un ciclo interactivo el conocer sus parte, pero igual manera es mas sencillo realizar ajustes al tener todo de una manera mas compacta.

después de este proyecto me quedan arraigadas en mis nuevas formas de darle manejo a la creación de un código, entiendo la importancia de un "*Docstrings*", ya que no es simplemente nombrar por nombrar funciones variables clases etc, muchas veces mientras se genera el código podemos caer no entender lo que estamos desarrollando cuando nuevamente hacemos una revisión de lo que hemos creado, algo tan sencillo como el nombre de una variable a secas nos puede perjudicar en tiempos muertos de entendimiento de nuestro código cuando queramos volver a revisarlo y realizar alguna clase de ajuste.

Bibliografía

Trejos Buriticá, O. I. (2023a). Capítulo 11: Funciones. En *Probabilidades y matemáticas aplicadas* (pp. 369–420). Ediciones de la U. <https://www-ebooks7-24-com.ucompensar.basesdedatosezproxy.com>

Trejos Buriticá, O. I. (2023b). Capítulo 12: Consejos y reflexiones sobre programación. En *Probabilidades y matemáticas aplicadas* (pp. 423–440). Ediciones de la U. <https://www-ebooks7-24-com.ucompensar.basesdedatosezproxy.com>

Trejos Buriticá, O. I. (2023c). Capítulo 9: Vectores. En *Probabilidades y matemáticas aplicadas* (pp. 239–308). Ediciones de la U. <https://www-ebooks7-24-com.ucompensar.basesdedatosezproxy.com>

Trejos Buriticá, O. I. (2023d). Capítulo 10: Matrices. En *Probabilidades y matemáticas aplicadas* (pp. 309–368). Ediciones de la U. <https://www-ebooks7-24-com.ucompensar.basesdedatosezproxy.com>